CELL 1 — MOUNT DRIVE

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

CELL 2 — IMPORTS

```
import os
import re
import cv2
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from glob import glob
from sklearn.metrics import confusion_matrix
from scipy.ndimage import map_coordinates, gaussian_filter
```

CELL 3 — PATHS

```
BASE="/content/drive/MyDrive/Project work/Dataset/lgg-mri-segmentation/lgg_structured"

MODEL_DIR="/content/drive/MyDrive/Project work/models/Segmentation"
os.makedirs(MODEL_DIR,exist_ok=True)

TRAIN_IMG=os.path.join(BASE,"train/images")
TRAIN_MASK=os.path.join(BASE,"train/masks")

VAL_IMG=os.path.join(BASE,"val/images")
VAL_MASK=os.path.join(BASE,"val/masks")

TEST_IMG=os.path.join(BASE,"test/images")
TEST_MASK=os.path.join(BASE,"test/masks")

CKPT="/content/best.h5"
SAVE_PATH=os.path.join(MODEL_DIR,"attention_resunet___.keras")

for name,p in [
("train_img",TRAIN_IMG),
("train_mask",TRAIN_MASK),
("val_img",VAL_IMG),
("val_mask",VAL_MASK),
("test_img",TEST_IMG),
("test_mask",TEST_MASK)
]:
    print(name,os.path.exists(p))
```

```
train_img True
train_mask True
val_img True
val_mask True
test_img True
test_mask True
```

CELL 4 — CONFIG

```
IMG_SIZE=256
BATCH_SIZE=8
EPOCHS=80
SEED=42
AUTOTUNE=tf.data.AUTOTUNE
```

CELL 5 — DATA LOADING

```
def nsort(p):
    return [int(n) for n in re.findall(r"\d+",os.path.basename(p))]

train_imgs=sorted(glob(os.path.join(TRAIN_IMG,"*")),key=nsort)
train_masks=sorted(glob(os.path.join(TRAIN_MASK,"*")),key=nsort)

val_imgs=sorted(glob(os.path.join(VAL_IMG,"*")),key=nsort)
val_masks=sorted(glob(os.path.join(VAL_MASK,"*")),key=nsort)

test_imgs=sorted(glob(os.path.join(TEST_IMG,"*")),key=nsort)
test_masks=sorted(glob(os.path.join(TEST_MASK,"*")),key=nsort)

print("Train:",len(train_imgs))
print("Val:",len(val_imgs))
print("Test:",len(test_imgs))

assert len(train_imgs)==len(train_masks)
assert len(val_imgs)==len(val_masks)
```

```
Train: 3133
Val: 409
Test: 387
```

CELL 6 — FILTER EMPTY MASKS

```
def has_tumor(mask_path):

    m=cv2.imread(mask_path,cv2.IMREAD_GRAYSCALE)

    return m is not None and m.max()>0


train_pairs=[(i,m) for i,m in zip(train_imgs,train_masks) if has_tumor(m)]
val_pairs=[(i,m) for i,m in zip(val_imgs,val_masks) if has_tumor(m)]

train_imgs_f,train_masks_f=map(list,zip(*train_pairs))
val_imgs_f,val_masks_f=map(list,zip(*val_pairs))

print("Filtered Train:",len(train_imgs_f))
print("Filtered Val:",len(val_imgs_f))
```

```
Filtered Train: 1065
Filtered Val: 151
```

CELL 7 — PREPROCESSING

```
def load_image(path):

    img=cv2.imread(path.numpy().decode(),cv2.IMREAD_GRAYSCALE)

    if img is None:
        return np.zeros((IMG_SIZE,IMG_SIZE,1),np.float32)

    img=cv2.resize(img,(IMG_SIZE,IMG_SIZE)).astype(np.float32)/255.0

    clahe=cv2.createCLAHE(clipLimit=2.0,tileGridSize=(8,8))
    img=clahe.apply((img*255).astype(np.uint8))

    img=img.astype(np.float32)/255.0

    return np.expand_dims(img,-1)


def load_mask(path):

    m=cv2.imread(path.numpy().decode(),cv2.IMREAD_GRAYSCALE)

    if m is None:
        return np.zeros((IMG_SIZE,IMG_SIZE,1),np.float32)

    m=cv2.resize(m,(IMG_SIZE,IMG_SIZE),interpolation=cv2.INTER_NEAREST)
    m=(m>127).astype(np.float32)

    return np.expand_dims(m,-1)
```

```
def parse(img_path,mask_path):

    img=tf.py_function(load_image,[img_path],tf.float32)
    mask=tf.py_function(load_mask,[mask_path],tf.float32)

    img.set_shape([IMG_SIZE,IMG_SIZE,1])
    mask.set_shape([IMG_SIZE,IMG_SIZE,1])

    return img,mask
```

CELL 8 — AUGMENTATION

```
def augment(img,mask):

    if tf.random.uniform(())>0.5:
        img=tf.image.flip_left_right(img)
        mask=tf.image.flip_left_right(mask)

    if tf.random.uniform(())>0.5:
        img=tf.image.flip_up_down(img)
        mask=tf.image.flip_up_down(mask)

    k=tf.random.uniform([],0,4,dtype=tf.int32)
    img=tf.image.rot90(img,k)
    mask=tf.image.rot90(mask,k)

    mask=tf.cast(mask>0.5,tf.float32)

    return img,mask


def to_dict(img,mask):
    return img, mask
```

CELL 9 — DATA PIPELINE

```
train_ds=(
tf.data.Dataset.from_tensor_slices((train_imgs_f,train_masks_f))
.shuffle(len(train_imgs_f),seed=SEED)
.map(parse,num_parallel_calls=AUTOTUNE)
.map(augment,num_parallel_calls=AUTOTUNE)
.map(to_dict,num_parallel_calls=AUTOTUNE)
.batch(BATCH_SIZE)
.prefetch(AUTOTUNE)
)

val_ds=(
tf.data.Dataset.from_tensor_slices((val_imgs_f,val_masks_f))
.map(parse,num_parallel_calls=AUTOTUNE)
.map(to_dict,num_parallel_calls=AUTOTUNE)
.batch(BATCH_SIZE)
.prefetch(AUTOTUNE)
)

print("datasets ready")
```

```
datasets ready
```

CELL 10 — LOSS & METRICS

```
def dice_coef(y_true,y_pred,smooth=1):

    y_true=tf.reshape(tf.cast(y_true,tf.float32),[-1])
    y_pred=tf.reshape(tf.cast(y_pred,tf.float32),[-1])

    inter=tf.reduce_sum(y_true*y_pred)

    return (2*inter+smooth)/(tf.reduce_sum(y_true)+tf.reduce_sum(y_pred)+smooth)


def combined_loss(y_true,y_pred):
```

```
    bce=tf.reduce_mean(tf.keras.losses.binary_crossentropy(y_true,y_pred))

    inter=tf.reduce_sum(y_true*y_pred)
    dice=(2*inter+1)/(tf.reduce_sum(y_true)+tf.reduce_sum(y_pred)+1)

    return bce+(1-dice)
```

CELL 11 — MODEL BLOCKS

```
def conv_block(x,f):

    x=tf.keras.layers.Conv2D(f,3,padding="same")(x)
    x=tf.keras.layers.BatchNormalization()(x)
    x=tf.keras.layers.Activation("relu")(x)

    x=tf.keras.layers.Conv2D(f,3,padding="same")(x)
    x=tf.keras.layers.BatchNormalization()(x)
    x=tf.keras.layers.Activation("relu")(x)

    return x


def res_block(inp,f):

    x=conv_block(inp,f)

    r=tf.keras.layers.Conv2D(f,1,padding="same")(inp)

    return tf.keras.layers.Add()([x,r])
```

CELL 12 — BUILD MODEL

```
def build_model(size):

    inp=tf.keras.Input((size,size,1))

    e1=res_block(inp,64)
    p1=tf.keras.layers.MaxPooling2D()(e1)

    e2=res_block(p1,128)
    p2=tf.keras.layers.MaxPooling2D()(e2)

    e3=res_block(p2,256)
    p3=tf.keras.layers.MaxPooling2D()(e3)

    b=conv_block(p3,512)

    u1=tf.keras.layers.Conv2DTranspose(256,2,strides=2,padding="same")(b)
    d1=conv_block(tf.keras.layers.Concatenate()([u1,e3]),256)

    u2=tf.keras.layers.Conv2DTranspose(128,2,strides=2,padding="same")(d1)
    d2=conv_block(tf.keras.layers.Concatenate()([u2,e2]),128)

    u3=tf.keras.layers.Conv2DTranspose(64,2,strides=2,padding="same")(d2)
    d3=conv_block(tf.keras.layers.Concatenate()([u3,e1]),64)

    out=tf.keras.layers.Conv2D(1,1,activation="sigmoid")(d3)

    return tf.keras.Model(inp,out)

model=build_model(IMG_SIZE)
model.summary()
```

```
Model: "functional_1"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 256, 256, 1)	0	-
conv2d_18 (Conv2D)	(None, 256, 256, 64)	640	input_layer_1[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 256, 256, 64)	256	conv2d_18[0][0]
activation_14 (Activation)	(None, 256, 256, 64)	0	batch_normalizat…
conv2d_19 (Conv2D)	(None, 256, 256, 64)	36,928	activation_14[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 256, 256, 64)	256	conv2d_19[0][0]
activation_15 (Activation)	(None, 256, 256, 64)	0	batch_normalizat…
conv2d_20 (Conv2D)	(None, 256, 256, 64)	128	input_layer_1[0]…
add_3 (Add)	(None, 256, 256, 64)	0	activation_15[0]… conv2d_20[0][0]
max_pooling2d_3 (MaxPooling2D)	(None, 128, 128, 64)	0	add_3[0][0]
conv2d_21 (Conv2D)	(None, 128, 128, 128)	73,856	max_pooling2d_3[…
batch_normalizatio… (BatchNormalizatio…	(None, 128, 128, 128)	512	conv2d_21[0][0]
activation_16 (Activation)	(None, 128, 128, 128)	0	batch_normalizat…
conv2d_22 (Conv2D)	(None, 128, 128, 128)	147,584	activation_16[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 128, 128, 128)	512	conv2d_22[0][0]
activation_17 (Activation)	(None, 128, 128, 128)	0	batch_normalizat…
conv2d_23 (Conv2D)	(None, 128, 128, 128)	8,320	max_pooling2d_3[…
add_4 (Add)	(None, 128, 128, 128)	0	activation_17[0]… conv2d_23[0][0]
max_pooling2d_4 (MaxPooling2D)	(None, 64, 64, 128)	0	add_4[0][0]
conv2d_24 (Conv2D)	(None, 64, 64, 256)	295,168	max_pooling2d_4[…
(Activation)	256)		
conv2d_25 (Conv2D)	(None, 64, 64, 256)	590,080	activation_18[0]…

CELL 13 — COMPILE

```
model.compile(
optimizer=tf.keras.optimizers.Adam(1e-4),
loss=combined_loss,
metrics=[dice_coef]
)
```

CELL 14 — CALLBACKS

```
callbacks=[

tf.keras.callbacks.ModelCheckpoint(
CKPT,
monitor="val_dice_coef",
mode="max",
save_best_only=True
),

tf.keras.callbacks.EarlyStopping(
monitor="val_dice_coef",
```

```
    mode="max",
    patience=15,
    restore_best_weights=True
    )

    ]
```

CELL 15 — TRAIN

```
| activation_20 (Activation) | (None, 32, 32, 512) | 0 | batch_normalizat… |
|---|---|---|---|

history=model.fit(
train_ds,
validation_data=val_ds,
epochs=EPOCHS,
callbacks=callbacks
)

| conv2d_transpose_3 (Conv2DTranspose) | (None, 64, 64, 256) | 524,544 | activation_21[0]… |
| concatenate_3 (Concatenate) | (None, 64, 64, 512) | 0 | conv2d_transpose… add_5[0][0] |
| conv2d_29 (Conv2D) | (None, 64, 64, 256) | 1,179,904 | concatenate_3[0]… |
| batch_normalizatio… (BatchNormalizatio… | (None, 64, 64, 256) | 1,024 | conv2d_29[0][0] |
| activation_22 (Activation) | (None, 64, 64, 256) | 0 | batch_normalizat… |
| conv2d_30 (Conv2D) | (None, 64, 64, 256) | 590,080 | activation_22[0]… |
| batch_normalizatio… (BatchNormalizatio… | (None, 64, 64, 256) | 1,024 | conv2d_30[0][0] |
| activation_23 (Activation) | (None, 64, 64, 256) | 0 | batch_normalizat… |
| conv2d_transpose_4 (Conv2DTranspose) | (None, 128, 128, 128) | 131,200 | activation_23[0]… |
| concatenate_4 (Concatenate) | (None, 128, 128, 256) | 0 | conv2d_transpose… add_4[0][0] |
| conv2d_31 (Conv2D) | (None, 128, 128, 128) | 295,040 | concatenate_4[0]… |
| batch_normalizatio… (BatchNormalizatio… | (None, 128, 128, 128) | 512 | conv2d_31[0][0] |
| activation_24 (Activation) | (None, 128, 128, 128) | 0 | batch_normalizat… |
| conv2d_32 (Conv2D) | (None, 128, 128, 128) | 147,584 | activation_24[0]… |
| batch_normalizatio… (BatchNormalizatio… | (None, 128, 128, 128) | 512 | conv2d_32[0][0] |
| activation_25 (Activation) | (None, 128, 128, 128) | 0 | batch_normalizat… |
| conv2d_transpose_5 (Conv2DTranspose) | (None, 256, 256, 64) | 32,832 | activation_25[0]… |
| concatenate_5 (Concatenate) | (None, 256, 256, 128) | 0 | conv2d_transpose… add_3[0][0] |
| conv2d_33 (Conv2D) | (None, 256, 256, 64) | 73,792 | concatenate_5[0]… |
| batch_normalizatio… | (None, 256, 256, | 256 | conv2d_33[0][0] |

Epoch 1/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 987ms/step - dice_coef: 0.1116 - loss: 1.3436WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 183s 1s/step - dice_coef: 0.1119 - loss: 1.3422 - val_dice_coef: 0.0566 - val_loss: 1.4480
Epoch 2/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - dice_coef: 0.2461 - loss: 0.9074WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 77ms/step - dice_coef: 0.2462 - loss: 0.9072 - val_dice_coef: 0.0585 - val_loss: 1.1966
Epoch 3/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step - dice_coef: 0.2917 - loss: 0.8378WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 77ms/step - dice_coef: 0.2919 - loss: 0.8375 - val_dice_coef: 0.1054 - val_loss: 1.1229
Epoch 4/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step - dice_coef: 0.3629 - loss: 0.7453WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 75ms/step - dice_coef: 0.3630 - loss: 0.7451 - val_dice_coef: 0.2254 - val_loss: 1.0488
Epoch 5/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - dice_coef: 0.4199 - loss: 0.6717WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 76ms/step - dice_coef: 0.4201 - loss: 0.6715 - val_dice_coef: 0.2384 - val_loss: 1.3186
Epoch 6/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - dice_coef: 0.4620 - loss: 0.6229WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - dice_coef: 0.4621 - loss: 0.6227 - val_dice_coef: 0.3658 - val_loss: 0.7451
Epoch 7/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 74ms/step - dice_coef: 0.5180 - loss: 0.5563 - val_dice_coef: 0.3576 - val_loss: 0.7310
Epoch 8/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - dice_coef: 0.5700 - loss: 0.4927WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 76ms/step - dice_coef: 0.5710 - loss: 0.4926 - val_dice_coef: 0.3916 - val_loss: 0.6865
Epoch 9/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step - dice_coef: 0.5899 - loss: 0.4721WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 75ms/step - dice_coef: 0.5901 - loss: 0.4720 - val_dice_coef: 0.4220 - val_loss: 0.6621
Epoch 10/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step - dice_coef: 0.6098 - loss: 0.4115WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 75ms/step - dice_coef: 0.6098 - loss: 0.4115 - val_dice_coef: 0.4624 - val_loss: 0.6630
Epoch 11/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 72ms/step - dice_coef: 0.6788 - loss: 0.3732 - val_dice_coef: 0.4043 - val_loss: 0.6880
Epoch 12/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step - dice_coef: 0.6598 - loss: 0.3942WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 75ms/step - dice_coef: 0.6599 - loss: 0.3940 - val_dice_coef: 0.5184 - val_loss: 0.5698
Epoch 13/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 73ms/step - dice_coef: 0.7006 - loss: 0.3494 - val_dice_coef: 0.5129 - val_loss: 0.5861
Epoch 14/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - dice_coef: 0.7067 - loss: 0.3438 - val_dice_coef: 0.4958 - val_loss: 0.5786
Epoch 15/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 73ms/step - dice_coef: 0.7139 - loss: 0.3346 - val_dice_coef: 0.4571 - val_loss: 0.6373
Epoch 16/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 72ms/step - dice_coef: 0.7234 - loss: 0.3237 - val_dice_coef: 0.4298 - val_loss: 0.6475
Epoch 17/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 72ms/step - dice_coef: 0.7569 - loss: 0.2873 - val_dice_coef: 0.4026 - val_loss: 0.6852
Epoch 18/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 73ms/step - dice_coef: 0.7714 - loss: 0.2717 - val_dice_coef: 0.5017 - val_loss: 0.5907
Epoch 19/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - dice_coef: 0.7441 - loss: 0.3050WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 76ms/step - dice_coef: 0.7441 - loss: 0.3049 - val_dice_coef: 0.5469 - val_loss: 0.5321
Epoch 20/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 74ms/step - dice_coef: 0.7647 - loss: 0.2778 - val_dice_coef: 0.4948 - val_loss: 0.7120
Epoch 21/80
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 72ms/step - dice_coef: 0.7821 - loss: 0.2609 - val_dice_coef: 0.5337 - val_loss: 0.5410
Epoch 22/80
133/134 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step - dice_coef: 0.7825 - loss: 0.2597WARNING:absl:You are saving your model as an HDF5
134/134 ━━━━━━━━━━━━━━━━━━━━ 10s 75ms/step - dice_coef: 0.7826 - loss: 0.2596 - val_dice_coef: 0.5526 - val_loss: 0.5305
Epoch 23/80
```

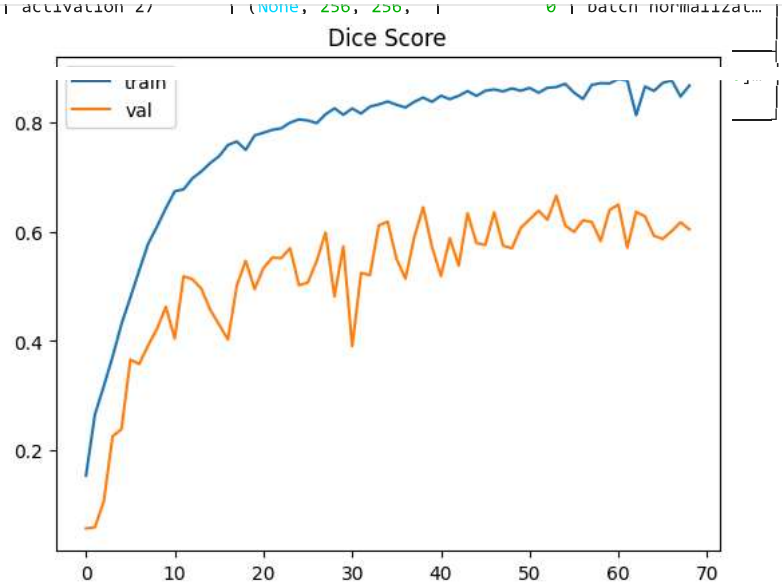
CELL 16 — GRAPH

```
| activation_26 (Activation) | (None, 256, 256, 64) | 0 | batch_normalizat… |

plt.plot(history.history["dice_coef"])
plt.plot(history.history["val_dice_coef"])
```

```
plt.title("Dice Score")
plt.legend(["train","val"])
```

```
activation_27        (None, 256, 256,          0  batch_normalizat…
```



CELL 17 — TEST DATASET

```
test_ds=(
tf.data.Dataset.from_tensor_slices((test_imgs,test_masks))
.map(parse)
.batch(BATCH_SIZE)
)
```

CELL 18 — EVALUATION

```
results=model.evaluate(test_ds)

print("Test Dice:",results[1])
```

```
49/49 ━━━━━━━━━━━━━━━━━━━━ 442s 9s/step - dice_coef: 0.3135 - loss: 0.7115
Test Dice: 0.3902296721935272
```

CELL 19 — SAVE MODEL

```
model.save(SAVE_PATH)

print("Model saved:",SAVE_PATH)
```

```
Model saved: /content/drive/MyDrive/Project work/models/Segmentation/attention_resunet___.keras
```